

From Molecular Computing to Molecular Programming

Masami Hagiya

Graduate School of Science, University of Tokyo

hagiya@is.s.u-tokyo.ac.jp

Project home page: <http://hagi.is.s.u-tokyo.ac.jp/MCP/>

Abstract. The purpose of this article is to survey the research in molecular computing, including the achievements of the Japanese Molecular Computer Project, and foresee the future of the field. In addition to describing the major achievements of the project, Suyama's *Dynamic Programming Molecular Computer* and Sakamoto's *Hairpin Engines*, we summarize the computational paradigms related to molecular computing in order to provide a perspective on the field. We finally explain the idea of *molecular programming*, that the author is currently advocating.

1 Introduction

The Japanese Molecular Computer Project, sponsored by the Japan Society for the Promotion of Science, began in October 1996 and will end in March 2001 [8]. The major achievements of the project include Suyama's *Dynamic Programming Molecular Computer*, which drastically reduces the number of molecules in Adleman-style DNA computations and allows for automation by robots, and Sakamoto's *Hairpin Engines*, which employ hairpin forms, the typical secondary structures taken by DNA, for molecular computations. These achievements are supported by a series of theoretical studies by Yokomori's theory group, Nishikawa's simulator for DNA computations, Arita's recent work on code design, etc.

In particular, Sakamoto's two types of Hairpin Engine, *Whiplash PCR* and *SAT Engine*, are of great importance to the community of molecular computing, primarily because essential computational steps are implemented by self-directed hairpin formation, and no operations are necessary outside the test tube. Hairpin Engines thus realize a kind of *autonomous molecular computation*. This work also suggests the importance of *programming* molecules. In order to control successive autonomous reactions, including secondary structure formation, we should go beyond simple code design and carefully *program* DNA sequences. We believe that the technology of programming molecules should be one of the central research issues in the next decade. If it is ever established, it will be applied not only to computation, but also to many areas of molecular science.

This article surveys the research in molecular computing, including the above-mentioned achievements of our project, and foresees the future of the field.

Molecular computing aims to analyze the computational power of biomolecules such as DNA and protein, and seek engineering applications of the analyses. Engineering applications are not restricted to computers; they include any system that has information processing capabilities at the level of molecules. Information processing in nanomachines, for instance, should inevitably be implemented by molecular computation. Molecular biology, pharmacy, and medicine are also considered possible application areas. In particular, the application of molecular computation to biotechnology has recently been called *computationally inspired biotechnology*.

This article starts with Adleman-style DNA computing, which we call the *Adleman-Lipton Paradigm*, and efforts to improve the paradigm, including *Dynamic Programming Molecular Computer*. We then introduce the field of *autonomous molecular computing*, which tries to analyze computations by molecules in pure forms, and describe the achievements of the field, including *Hairpin Engines*. In order to provide a wider perspective of molecular computing, we also summarize the related computational paradigms. They include H-system, P-system, concurrency calculi, cellular computing, amorphous computing, etc. In the last section, the concept of *molecular programming*, which the author is currently advocating, is briefly described.

2 Improvements of the Adleman-Lipton Paradigm

In 1994, Adleman reported in *Science* a method of solving the Hamiltonian path problem using DNA molecules, and an experiment in which he actually solved the problem for a 7-vertex directed graph by the method [2]. Lipton generalized Adleman's method and proposed an approach to solving NP-complete problems by DNA molecules, in particular, the satisfiability problem of Boolean formulas (the SAT problem) [11]. In this article, we call this approach to DNA computing, proposed by Adleman and established by Lipton, the *Adleman-Lipton Paradigm*.

The Adleman-Lipton Paradigm solves combinatorial optimization problems, such as NP-complete problems, using DNA molecules. Each solution candidate of a problem is represented by a DNA molecule. The paradigm consists of the following two steps.

- (1) Solution candidates are randomly generated.
- (2) Real solutions are selected from among the generated candidates.

In the first step, the ability of DNA molecules to hybridize with one another through Watson-Crick base-pairing is exploited. In Adleman's experiment, vertices and edges in a directed graph are represented by single-stranded DNA molecules. They hybridize together to form a double-stranded DNA molecule that represents a path in the graph.

In the second step, real solutions are selected by molecular biology operations. Each operation is applied to all the candidates in a test tube. This step is a typical data-parallel computation.

The complexity of these molecular computations can be measured by the required time and the number of necessary molecules. Since the Adleman-Lipton paradigm reduces the time by sacrificing the number of molecules, the size of problems solvable by this method is limited by the number of molecules required for computations. So, the crucial point of the paradigm is how to reduce the number of potential candidates.

Many improvements upon the Adleman-Lipton paradigm have been proposed. Among others, Suyama et al. have proposed an iterative approach. Rather than generate all of the random candidates prior to selection, generation and selection of partial solutions is repeated [12]. In this approach, once candidates of partial solutions are identified, those that cannot be extended to a total solution are immediately removed. The remaining partial solutions are then extended, and the process is reapplied to these extended partial solutions.

Utilizing this approach, Suyama et al. designed an algorithm for solving the satisfiability problem of 3-literal clausal formulas (the 3-SAT problem) [25]. A similar algorithm had been independently proposed by Ogihara and Ray [14], but Suyama et al. actually implemented their algorithm using DNA molecules. The algorithm does not generate assignments of all the variables at once. Rather it extends partial assignments for one variable at a time. Those partial assignments that cannot be extended to complete satisfying assignments are removed.

Using this algorithm, they succeeded in solving a 4-variable 10-clause instance of the 3-SAT problem [25]. Currently, they are developing a DNA computing robot, shown in Fig. 1, with which they plan to solve a 30-variable 100-clause instance of the problem. According to Suyama's estimation, solving a 100-variable instance is not impossible. However, even if a 100-variable instance of the 3-SAT problem could be solved, DNA computers would not outperform electronic computers. Breakthroughs are required both in experimental technologies and in algorithms, before molecular computers based on the Adleman-Lipton paradigm can solve problems that are beyond the capability of electronic computers.

3 Autonomous Molecular Computing

In the first step of the Adleman-Lipton paradigm, solution candidates are generated by the ability of DNA molecules to hybridize together. In this step, the computation proceeds without any operations outside the test tube. On the other hand, in the second step, a large number of laboratory operations are applied from outside the tube.

Molecular computations are called *autonomous* if they proceed by successive autonomous reactions of molecules. The first step of the Adleman-Lipton paradigm is a typical example of autonomous molecular computation. They are also called *one-pot reactions*, because they autonomously proceed in a single test tube once the necessary ingredients are put into the tube initially. Research on autonomous molecular computation is considered to analyze the computational power of molecules in pure forms. In the rest of this section, we describe Sakamoto's research after briefly touching upon Winfree's.



Fig. 1. Suyama's robot.

3.1 Winfree's DNA Tiles

Hybridization of DNA molecules is also called *self-assembly*, because DNA molecules autonomously assemble together. The computational power of self-assembly by DNA molecules has been thoroughly investigated by Winfree [23]. In particular, he pointed out the computational power of DNA molecules called *DNA tiles*. *Double-crossover molecules* are typical examples of DNA tiles (Fig. 2, lower left). These are structures made of two double-stranded molecules which exchange their single strands at two points. Each double-crossover molecule is composed of four single strands that self-assemble.

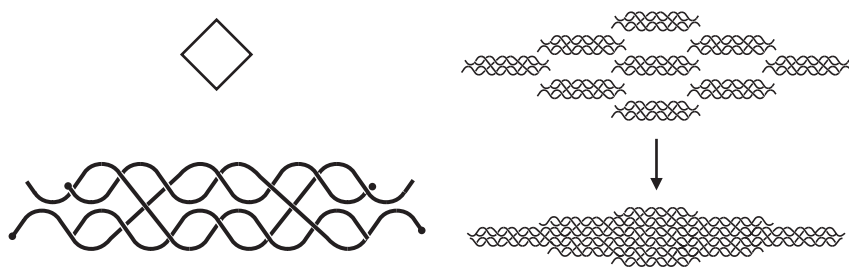


Fig. 2. Winfree's DNA tiles.

Double-crossover molecules can also self-assemble together and form a planar structure [22], because they have four sticky ends, each hybridizing with a sticky end of another molecule (Fig. 2, right). This process of self-assembly corresponds to that of square tiles that have colored edges and can hybridize only if adjacent edges are of the same color (Fig. 2, upper left).

The tiling process of square tiles with colored edges is known to have high computational power [23]. For instance, it can simulate execution of one-dimensional cellular automata, and thus has universal computability.

3.2 SAT Engine

Sakamoto and his group, which includes the author, have been investigating computational schemes employing hairpin forms of DNA molecules. So far, they have proposed two computational schemes and have finished preliminary experiments for their implementation. Hairpin forms are the most fundamental of the secondary structures taken by DNA molecules. In this sense, the work of Sakamoto et al. is considered the first step towards establishing computations employing molecular structures. Sakamoto calls these computational schemes based on hairpin formation *Hairpin Engines*.

SAT Engine, recently reported in *Science*, is a computational scheme for solving the satisfiability problem of clausal formulas [19]. Since a clausal formula is a conjunction of clauses, to satisfy such a formula is to satisfy all of its clauses. Since a clause is a disjunction of literals, to satisfy a clause is to satisfy at least one of its literals. Therefore, in order to satisfy a clausal formula, it suffices to select one literal from each clause and compose an assignment by the selected literals. For instance, if literal a is selected from the clause, $(a \vee \neg b \vee c)$, then a is made true in the composed assignment. If $\neg b$ is selected, b is made false. If a variable and its negation are both selected (from different clauses), it is impossible to compose an assignment since they are inconsistent. On the other hand, if inconsistent literals are not selected, one can obtain a satisfying assignment.

SAT Engine is based on the idea to represent complementary literals, i.e., a variable and its negation, by Watson-Crick complementary sequences of DNA. The literals selected from the clauses are concatenated into one single-stranded DNA molecule. If the selection contains inconsistent literals, the molecule contains complementary sequences and forms a hairpin structure (Fig. 3). If the selection is consistent, the molecule does not form a hairpin structure. Therefore, it is possible to decide whether a clausal formula is satisfiable by generating a random pool of molecules, each composed of literals selected from clauses, and distinguishing the molecules with hairpins from those without hairpins.

SAT Engine uses the following two methods to distinguish hairpins from non-hairpins.

- (1) The recognition site of a restriction enzyme is inserted into the middle of a sequence encoding a literal. Molecules with hairpins are cut by the enzyme.
- (2) The efficiency of amplification by PCR differs for molecules with hairpins and without hairpins. The former kind of molecule is less efficient in amplification than the latter. Sakamoto developed a new kind of PCR, called

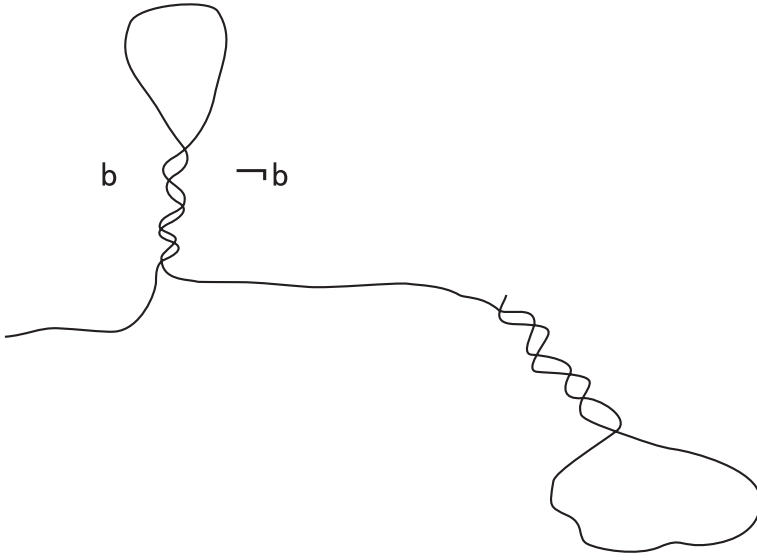


Fig. 3. Inconsistent assignment.

exclusive PCR (*ePCR*), which exaggerates the difference between the two kinds of molecules.

Using the above two methods, molecules with hairpins and molecules without hairpins can be separated. Although this process does require a number of laboratory operations, the most important part of the computation, the detection of inconsistent assignments, is realized by hairpin formation, which is an autonomous reaction of the molecules. Note also that the number of laboratory operations is not dependent on the number of variables or clauses. Sakamoto et al. actually solved a 6-variable 10-variable instance of the 3-SAT problem [19].

3.3 Whiplash PCR

Whiplash PCR is a method to realize state machines using DNA molecules [7]. Sakamoto et al. have been trying to implement state machines that have internal programs, using DNA molecules. Remember that the Adleman-Lipton Paradigm is a computational scheme for data-parallel computations, in which programs are realized as sequences of laboratory operations. In contrast to the Adleman-Lipton Paradigm, Sakamoto et al. tried to represent programs by the molecules themselves. If each molecule has its own program, program-parallel computations can be implemented.

Whiplash PCR is based on the idea that the 3'-end of a single-stranded DNA molecule encodes the current state of the machine implemented by the

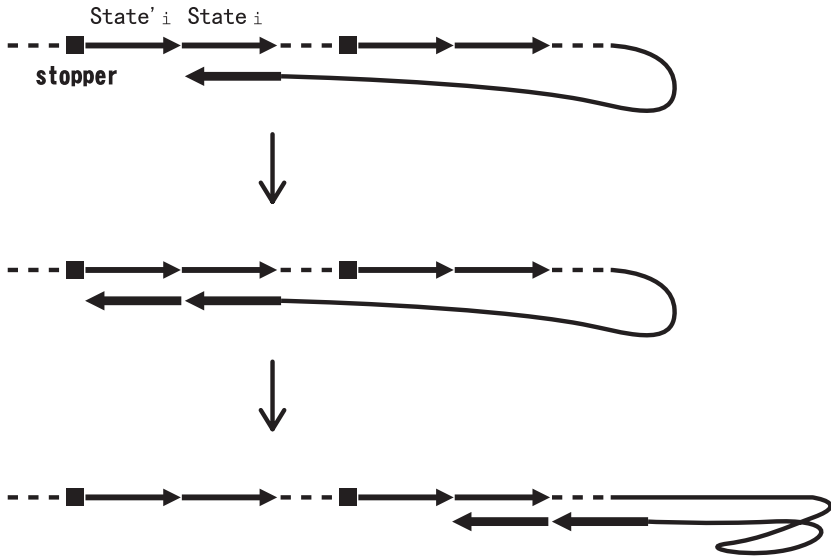


Fig. 4. Whiplash PCR.

molecule. It also contains a table for state transitions, which is the program of the machine. As in Fig. 4, the state transition table is a repetition of sequences, each of which is a concatenation of the stopper sequence (**stopper**), the state before the transition (State'_i) and the state after the transition (State_i).

By taking a hairpin form, the 3'-end hybridizes with some before-state (State_i) in the transition table of that molecule. This molecule is extended by the DNA polymerase. As a consequence, the complement of the after-state (State'_i) is attached to the 3'-end. This means that the current state is changed from State_i to State'_i . As in the figure, if the hairpin structure is denatured and another hairpin is formed, the next transition can occur. It is reported that several (about five) transitions have been experimentally confirmed [10].

The stopper sequence leads to cessation of extension by the DNA polymerase. There are many methods of implementing the stopper sequence. One that has been implemented is to have only three of the four possible deoxyribonucleotide triphosphates in the reaction buffer, and represent the stopper sequence as a polynucleotide of the base that is complementary to the one that is absent from the reaction mixture.

By appropriately designing transition tables, it is possible to implement many kinds of computation by Whiplash PCR. Sakamoto et al. proposed a computational scheme in which both a program that evaluates a μ -formula (read-once Boolean formula) and an input to the program are put on a single DNA molecule. This molecule then behaves as an independent computational unit and computes the value of the formula as a function of the input [7].

4 New Computational Paradigms

Around molecular computing, a number of new computational paradigms are emerging. These include:

- DNA computing
- molecular computing
- chemical computing
- aqueous computing
- crystal computing
- cell computing
- gel computing
- amorphous computing

These computational paradigms can be classified within the context of molecular computing into four kinds of computation:

- (1) computing inside a single molecule
- (2) computing by interactions among molecules
- (3) computing with membranes
- (4) computing with geometry

Note that each group is a source of great computational power. Also, there are connections between these groups.

4.1 Computing Inside a Single Molecule

Hairpin Engines are typical examples of computation inside a single molecule. In both SAT Engine and Whiplash PCR, computation is implemented by conformational change or structural formation of a DNA molecule.

Protein folding is also a typical example of structural formation by a single molecule. Fraenkel was the first to explicitly state the relationship between protein folding and computation [6]. He showed that protein folding is NP-complete by reducing the energy minimization problem of spin glasses, which was known to be NP-complete, into the protein folding problem. This result, however, is purely theoretical and there is a big gap between Fraenkel's model and an actual protein.

There are some proposals to implement nano-scale memory by partial modification of a single molecule. Although such proposals do not provide universal computational schemes, they would become the first practical molecular computers. In the *Stickers Model* of Roweis et al., read/write memory is implemented by attaching short DNA molecules, called *stickers*, to a long single-stranded DNA molecule [17].

Head and Yamamura showed that even write-once memory has high computational power [9]. They proposed *Aqueous Computing*, in which write-once memory is used to solve NP-complete problems such as the max clique problem. In their current approach, write-once memory is implemented by a plasmid

containing sequences called *stations*, each corresponding to one bit. A station is surrounded by the recognition sites of the same restriction enzyme, and writing on a station is implemented by removing the station from the plasmid through sequential restriction endonuclease digestion and ligation.

4.2 Computing by Interactions among Molecules

Typical examples of computation by molecular interactions are those by hybridization of DNA molecules. Almost all the computational schemes proposed in the field of DNA computing employ DNA hybridization. Hybridization of DNA is also called self-assembly, because DNA molecules autonomously assemble together.

Winfree characterized the computational power of self-assembly for various forms of DNA molecules [23]. He showed that the set of double-stranded molecules that can be formed by self-assembly from a finite set of single-stranded molecules is a regular language. If branching is allowed in double-stranded molecules, the power of self-assembly is increased to that of context-free languages, because branching molecules can represent structures corresponding to the derivation trees of a context-free grammar. Furthermore, the self-assembly of DNA tiles is universally computable (Section 3.1).

The *splicing system* introduced by Head, also known as the *H-system*, is a formal model of recombination of DNA, i.e., cutting by restriction enzymes and concatenation by ligase [16]. Long before Adleman's work, Head had modeled reactions of restriction enzymes and ligase, which are now employed in many computational schemes of DNA computing. So far, a huge number of theoretical studies on the H-system have been done from the perspective of formal language theory.

Abstract chemistry is a field in *artificial life* that investigates molecular interactions in terms of artificial chemical systems. A large number of abstract chemical systems have been proposed and analyzed by simulations (e.g., [4]). Studies on cellular automata can also be classified in the same research direction.

Computational schemes involving combinations of self-assembly and conformational change of molecules have also been proposed. It is known that by allowing molecules to change their structures, the computational power of self-assembly increases [24,18].

4.3 Computing with Membranes

Membranes are the most basic device for dividing solutions into separate partitions. Since solutes cannot move across a membrane, independent reactions can occur in each partition of the solution. A partition divided by a membrane is called a *compartment*.

Several methods for implementing membranes are known. *Mechanical-electrical micro-systems (MEMS)*, or chemical IC, are small chemical plants

implemented on a chip consisting of small reaction chambers, each considered to be a compartment. Liposomes are small spheres made of lipid bilayers that separate solutions into an inside and outside. There are some proposals for computing with liposomes, but their implementation is not easy.

A cell is the most typically encountered compartment. Many ideas have been proposed for computing with living cells. Reactions in a cell that can be employed for computation include metabolism, gene regulation and signal transduction. Creation of cell machines by artificially regulating such reactions has been proposed [21].

Many formal models have been proposed for computing with membranes. Concurrency calculi, such as *chemical abstract machines* [5], are examples of such models. So far, many calculi, including π -calculus, join calculus, and ambient calculus, have been proposed, but they are not strongly related to actual chemical reactions.

The *P-system*, proposed by Păun, is also a formal model for computing with membranes [15]. Strings are rewritten inside a membrane. Deletion of a membrane is also regulated by a rewrite rule. Like the H-system, the P-system has been investigated from the perspective of formal language theory.

4.4 Computing with Geometry

Computations with a membrane employ the very simple topology consisting of the inside and outside of the membrane. Computations with a richer topology, including 2-D or 3-D geometry, are worth investigating. For instance, each DNA tile in the entire structure has a 2-D or 3-D coordinate. Computations employing the coordinate of each tile would be very powerful.

Amorphous computing, proposed by Abelson et al., is a computational paradigm with computational particles scattered in a 2-D or 3-D field in an amorphous fashion [1]. Since it does not assume regular crystal structures, it would have many applications.

Each computational particle can emit transmission substances that diffuse and reach another particle, which senses the substances, changes its internal state, and emits its own transmission substances. Abelson et al. developed algorithms for computational particles to acquire positional information and create patterns. They even developed a programming language for describing the patterns. To implement amorphous computing, they propose *cellular computing* [17].

Amorphous computing is independent from molecular computing, but there are strong relationships between the two ideas. For example, it would be possible to implement amorphous computing with DNA tiles. Gel can also be employed for amorphous computing.

5 Molecular Programming

In molecular computing, engineering viewpoints are considered very important, because they distinguish molecular computing from related research fields such

as mathematical biology, complex systems, artificial life, etc. The ultimate goal of molecular computing should be to synthesize artificial information processing systems at the molecular level. To achieve this goal, new computational paradigms, new algorithms, and new programming languages need to be developed. We use the phrase, *molecular programming*, to explicitly denote research into this synthetic approach.

Molecular programming is a field that aims to *program* reactions of biomolecules. Programs that control reactions of biomolecules are classified into those that are encoded in molecules themselves, and those that are implemented as sequences of laboratory operations. The former kind of program is characteristic to biomolecules that have combinatorial complexity and autonomous computational power. By coordinating the former and latter kinds of programs, it should be possible to control reactions of biomolecules and thus realize molecules or molecular systems with the intended functions and structures.

In a sense, molecular programming is everywhere. In PCR, for instance, it is important to design good primer sequences, and this is an example of the first kind of programming. In addition, choosing appropriate reaction conditions, such as temperature, salt concentration, etc., is also important, and is the second kind of programming. Only by coordinating the two programs, it is possible to have efficient amplification.

Another example of the first kind of programming is the code design of DNA sequences in molecular computing. This problem is related to the design of PCR primers, and it has been actively investigated in the field of molecular computing (e.g., [3]). As the second kind of programming, we can classify research on modeling and simulating chemical reactions (e.g., [13]).

A more sophisticated example of molecular programming is that of DNA chips. Suyama has been developing a universal DNA chip [20], where mRNA transcripts are first converted to standard DNA sequences, which Suyama calls *DNA coded numbers* (DCN's), and the DCN's are then hybridized with oligomers attached to the DNA chip (Fig. 5). The process to convert mRNA transcripts to DCN's employs template sequences that hybridize with both the mRNA transcripts and the DCN's.

By changing the set of template sequences, it is possible to change the conversion and thus change the set of mRNA that can be detected by the chip. In this sense, the chip is universal.

Based on the universal chip, we can further develop a more intelligent DNA chip. Rules for genetic diagnoses are in general of the following form:

If gene A is expressed, gene B is not expressed and gene C is expressed,
then there is a danger of disease D.

This kind of rule can be naturally represented by a Boolean formula (or decision diagram). Since a large number of methods for evaluating Boolean formulas using molecules have been proposed, including Whiplash PCR, it is possible to make such diagnoses by DNA computation.

In Suyama's universal chip, mRNA transcripts are converted to DCN's. Therefore, DCN's can be directly used for computation. Using Whiplash PCR,

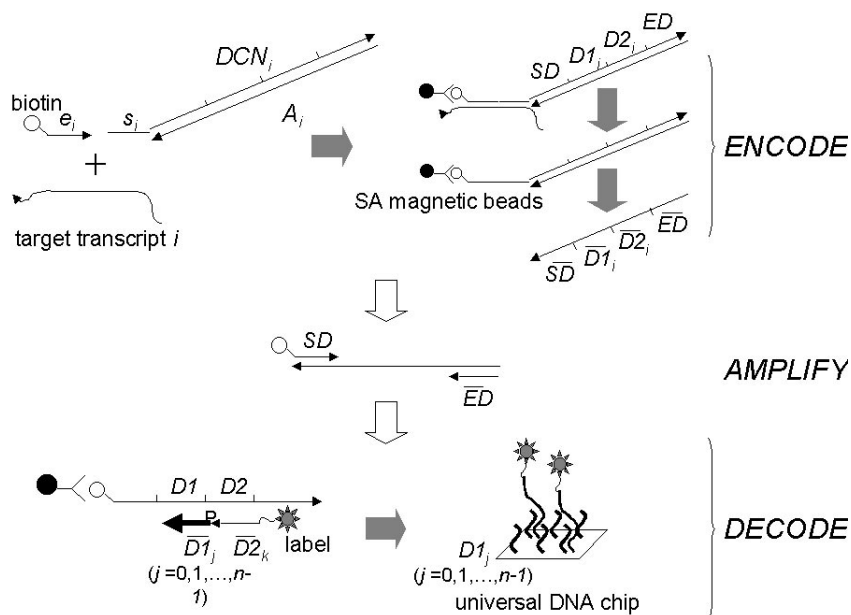


Fig. 5. Suyama's universal DNA chip.

even rules for diagnoses can be represented by molecules, so all the information processing can be done in a test tube.

The advantages of this intelligent DNA chip are:

- The mRNA transcripts need not be sequenced.
- Less information is revealed using a digital form. The data on mRNA transcripts are not revealed. In some cases, the rules for diagnoses can also be kept secret.

The above-mentioned intelligent DNA chips are one of the anticipated applications of molecular programming. Because of its fundamental nature, the application areas of molecular programming are considered broad and to cover various fields related to biomolecules, including genetic analysis, nanotechnology, nanomachine engineering, and combinatorial chemistry.

Acknowledgements. We deeply thank all the members of the Japanese Molecular Computer Project, which is supported by the Japan Society for the Promotion of Science under grand JSPS-RFTF 96I00101.

References

1. Harold Abelson, Don Allen, Daniel Coore, Chris Hanson, George Homsy, Thomas F. Knight, Jr., Radhika Nagpal, Erik Rauch, Gerald Jay Sussman, and Ron Weiss: Amorphous Computing, *Communications of the ACM*, Vol.43, No.5, pp.74–82, 2000.
2. Leonard M. Adleman: Molecular Computation of Solutions to Combinatorial Problems, *Science*, Vol.266, pp.1021–1024, 1994.
3. Masanori Arita, Akio Nishikawa and Masami Hagiya: Improving Sequence Design for DNA Computing, *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2000*, July 10–12, Las Vegas, Nevada, pp.875–882, 2000.
4. Wolfgang Banzhaf, Peter Dittrich and Burkart Eller: Selforganization in a system of binary strings with topological interactions, *Physica D*, Vol.125, pp.85–104, 1999.
5. Gérard Berry and Gérard Boudol: The Chemical Abstract Machine, *the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1990, pp.81–94. *Theoretical Computer Science*, Vol.96, 1992, pp.217–248.
6. Aviezri S. Fraenkel: Protein Folding, Spin Glass and Computational Complexity, *DNA Based Computers III, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, Vol.48, pp.101–121, 1999.
7. Masami Hagiya, Masanori Arita, Daisuke Kiga, Kensaku Sakamoto and Shigeyuki Yokoyama: Towards Parallel Evaluation and Learning of Boolean μ -Formulas with Molecules, *DNA Based Computers III, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, Vol.48, pp.57–72, 1999.
8. Masami Hagiya: Perspectives on Molecular Computing, *New Generation Computing*, Vol.17, No.2, pp.131–140, 1999.
9. Tom Head, Masayuki Yamamura and Susannah Gal: Aqueous Computing: Writing on Molecules, *Congress on Evolutionary Computation*, July 6–9, 1999, Mayflower Hotel, Washington D.C., USA, pp.1006–1010, 1999.
10. Ken Komiya, Kensaku Sakamoto, Hidetaka Gouzu, Shigeyuki Yokoyama, Masanori Arita, Akio Nishikawa and Masami Hagiya: Successive State Transitions with I/O Interface by Molecules, *DNA6, Sixth International Meeting on DNA Based Computers*, Leiden Center for Natural Computing, June 13–17, 2000, pp.1–30, 2000.
11. Richard J. Lipton: DNA Solution of Hard Computational Problems, *Science*, Vol.268, pp.542–545, 1995.
12. Nobuhiko Morimoto, Masanori Arita and Akira Suyama: Solid Phase DNA Solution to the Hamiltonian Path Problem, *DNA Based Computers III, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, Vol.48, pp.193–206, 1999.
13. Akio Nishikawa, Masayuki Yamamura and Masami Hagiya: DNA Computation Simulator Based on Abstract Bases, *Soft Computing*, to appear, 2000.
14. Mitsunori Ogihara and Animesh Ray: DNA-Based Parallel Computation by “Counting”, *DNA Based Computers III, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, Vol.48, pp.255–264, 1999.
15. Gheorghe Păun: Computing with Membranes, *TUCS Research Report*, No.208, November 1998, www.tucs.fi.
16. G. Păun, G. Rozenberg, and A. Salomaa: *DNA Computing*, Springer, 1998.
17. Sam Roweis, Erik Winfree, Richard Burgoyne, Nickolas V. Chelyapov, Myron F. Goodman, Paul W. K. Rothmund, and Leonard M. Adleman: A Sticker Based Model for DNA Computation, *DNA Based Computers II, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, Vol.44, pp.1–29, 1999.

18. Kazuhiro Saitou: Self-Assembling Automata: A Model of Conformational Self-Assembly, *Pacific Symposium on Biocomputing'98*, pp.609–620, 1998.
19. Kensaku Sakamoto, Hidetaka Gouzu, Ken komiya, Daisuke Kiga, Shigeyuki Yokoyama, Takashi Yokomori and Masami Hagiya: Molecular Computation by DNA Hairpin Formation, *Science*, Vol.288, pp.1223–1226, 2000.
20. Akira Suyama, Nao Nishida, Ken-ichi Kurata, Katsumi Omagari: Gene Expression Analysis by DNA Computing, *Currents in Computational Molecular Biology* (ISBN 4-946443-61-4), pp.12–13, 2000.
21. Ron Weiss and Thomas F. Knight, Jr.: Engineered Communications for Microbial Robotics, *DNA6, Sixth International Meeting on DNA Based Computers*, Leiden Center for Natural Computing, June 13–17, 2000, pp.5–19, 2000.
22. Erik Winfree, Furong Liu, Lisa A. Wenzler, and Nadrian C. Seeman: Design and self-assembly of two-dimensional DNA crystals, *Nature*, Vol.394, pp.539–544, 1998.
23. Erik Winfree, Xiaoping Yang and Nadrian C. Seeman: Universal Computation via Self-assembly of DNA: Some Theory and Experiments, *DNA Based Computers II, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, Vol.44, pp.191–213, 1999.
24. Takashi Yokomori: YAC: Yet Another Computation Model of Self-Assembly, *Preliminary Proceedings of the Fifth International Meeting on DNA Based Computers*, June 14–15, 1999, Massachusetts Institute of Technology, pp.153–167, 1999.
25. Hiroshi Yoshida and Akira Suyama: Solutions to 3-SAT by Breadth First Search, *Preliminary Proceedings of the Fifth International Meeting on DNA Based Computers*, June 14–15, 1999, Massachusetts Institute of Technology, pp.9–20, 1999.